NexaCorp Enterprise Knowledge Copilot — Product User Guide

Part 1: Setting Everything Up (One-Time Setup)

This section walks you through getting the entire system running on your machine from zero. You only need to do this once.

1.1 — What You Need Before Starting

Make sure these four things are installed on your Linux machine:

Software	What It Does	How to Check
Git	Downloads the project code	`git --version`
Python 3.11	Runs the application	`python3.11 --version`
Podman (or Docker)	Runs the Elasticsearch database in a container	`podman --version` or `docker --version`
Ollama	Runs the local AI model on your machine	`ollama --version`

If Ollama is not installed, run this single command:

```
curl -fsSL https://ollama.com/install.sh | sh
```

1.2 — Download the Project

Open a terminal and run:

```
git clone https://github.com/ichhit-ai/enterprise-knowledge-copilot.git
cd enterprise-knowledge-copilot
```

You now have the entire project on your machine.

1.3 — Install Python Libraries

Run these two commands inside the project folder:

```
pip install -r requirements.txt
```

This installs everything the app needs: the AI frameworks, the search engines, the web interface, and the privacy tools.

1.4 — Download the AI Model

This downloads the LLaMA 3.2 brain (~2 GB) to your machine. You only do this once:

```
ollama pull llama3.2
```

1.5 — Start the Elasticsearch Database

Elasticsearch is the high-performance search engine that powers the production mode. We run it inside a container so you don't need to install anything extra.

Option A — Using the included docker-compose file (easiest):

```
docker-compose up -d
```

Option B — Manual command:

```
podman run -d --name elasticsearch \
  -p 9200:9200 -p 9300:9300 \
  -e "discovery.type=single-node" \
  -e "xpack.security.enabled=false" \
  docker.elastic.co/elasticsearch/elasticsearch:8.11.1
```

How to verify it's working:

```
curl http://localhost:9200
```

You should see a JSON response that says `"tagline" : "You Know, for Search"`. If you see that, Elasticsearch is ready.

1.6 — Build the Search Databases (Data Ingestion)

This is the step where the app reads your 200,000 support tickets, strips out private information (names, emails, phone numbers), and builds searchable indexes.

Build the local ChromaDB + BM25 index:

```
PYTHONPATH=. python src/ingest_full.py
```

Takes about 3–5 minutes. Creates the `.index/` and `.index_full/` folders.

Build the Elasticsearch index:

```
PYTHONPATH=. python scripts/ingest_elasticsearch.py
```

Uploads all 200,000 tickets into your local Elasticsearch container.

After this step, your system is fully loaded and ready to answer questions.

1.7 — Start the Ollama AI Server

In a **separate terminal window**, start the AI server:

```
ollama serve
```

Leave this terminal open. The AI model needs this running to answer questions.

Part 2: Running the Application

You have **two ways** to use the copilot. Both connect to the same data and the same AI model.

2.1 — The Streamlit Dashboard (Visual, Interactive)

Start it:

```
streamlit run src/app_full.py
```

Open it: Go to `http://localhost:8501` in your browser.

What you'll see:

The screen is split into two sections:

Left Sidebar — Your Controls:

- **Role Selector**: Switch between Employee, Manager, or IT Admin. This changes what data you're allowed to see.
- **Engine Selector**: Choose between "Edge Sandbox" (local ChromaDB) or "Enterprise Cluster" (Elasticsearch). Both search the same 200,000 tickets, but Elasticsearch is faster at scale.
- **Show Reasoning Trace**: A checkbox that reveals the AI's step-by-step thought process.

Main Area — The Chat:

- Type any question into the chat box at the bottom.
- The AI will respond with an answer, pulled directly from your company's handbooks and ticket history.
- Below the answer, you'll see a 📄 **Sources** button. Click it to see the exact documents the AI used.
- Three quality scores appear under every answer:
  - **Confidence** — How closely the sources matched your question.
  - **Faithfulness** — Whether the AI only used facts from the documents (1.0 = no hallucination).
  - **Relevance** — How well the retrieved documents relate to your question.

2.2 — The FastAPI Backend (Fast, Programmatic)

Start it:

```
uvicorn fastapi_sandbox.main:app --host 0.0.0.0 --port 8000
```

Open it: Go to `http://localhost:8000` in your browser.

What you'll see:

- A clean web interface where you can type queries and get instant JSON responses.
- Go to `http://localhost:8000/docs` for the full interactive API documentation (Swagger UI) where you can test every endpoint.

Why use this instead of Streamlit?

- It's stateless — uses almost no RAM (120 MB vs Streamlit's 1.6+ GB).
- It handles hundreds of simultaneous requests without slowing down.
- Other applications and scripts can call it programmatically via REST API.

Part 3: Using the Features

3.1 — Asking Questions

Just type naturally. Here are some example queries that showcase different capabilities:

What You Type	What the AI Does
"What is the VPN access policy?"	Searches the company handbook
"Show me all P1 tickets from this month"	Filters tickets by priority using metadata
"How do I fix ERR-AUTH-9092?"	Finds the exact error code across runbooks and past tickets
"Who manages the AUTH-GATEWAY system?"	Queries the organizational knowledge graph
"Give me a summary of recent network outages"	Searches across all data sources and synthesizes a summary
"Create a ticket for the VPN being down"	Files a new support ticket (with duplicate detection)

3.2 — Privacy Protection (PII Redaction)

Every document goes through a privacy filter before it's stored. When you click 📄 **Sources** and expand the retrieved text, you'll notice:

- Real employee names are replaced with `[REDACTED_PERSON]`
- Email addresses are replaced with `[REDACTED_EMAIL]`
- Phone numbers are replaced with `[REDACTED_PHONE]`

This happens automatically. No sensitive personal data is ever stored in the search database.

3.3 — Smart Ticket Creation

When you ask the AI to create a ticket (e.g., *"File a ticket for the email server being down"*):

1. The AI first searches all existing open tickets.
2. If it finds a ticket that's more than **75% similar** to your request, it will **refuse to create a duplicate**.
3. Instead, it shows you: ⚠️ `Potential duplicate detected!` and links you to the existing ticket.
4. If no duplicate exists, it creates the ticket and confirms.

3.4 — Role-Based Access

Role	What You Can See
Employee	Public handbooks, general runbooks, non-sensitive ticket history

Manager	Everything above + org chart data, system ownership, escalation paths
IT Admin	Full access — security logs, restricted network data, ticket creation

Switch roles using the dropdown in the sidebar to test different access levels.

3.5 — Reasoning Trace

Check the **Show Reasoning Trace** box in the sidebar. After each answer, an expandable section will appear showing:

- Which tool the AI chose (e.g., `tool_search_docs` vs. `tool_search_tickets`)
- Why it chose that tool
- The full routing logic

This is useful for debugging and for demonstrating the AI's decision-making to stakeholders.

Part 4: Stopping and Restarting

To stop everything:

1. Press `Ctrl+C` in the terminal running Streamlit or FastAPI.
2. Press `Ctrl+C` in the terminal running `ollama serve`.
3. Stop the Elasticsearch container: `podman stop elasticsearch`

To restart later:

1. Start Elasticsearch: `podman start elasticsearch`
2. Start Ollama: `ollama serve`
3. Start the app: `streamlit run src/app_full.py` or `uvicorn fastapi_sandbox.main:app --port 8000`

You do **not** need to re-run the ingestion scripts. The data is already saved on disk.

Part 5: Troubleshooting

Problem	Solution
`ModuleNotFoundError: No module named 'spacy'`	Run `pip install -r requirements.txt` again
`ValueError: numpy.dtype size changed`	Run `pip install "numpy<2"` — this fixes a version conflict
`ConnectionError: Elasticsearch`	Run `podman start elasticsearch` and wait 10 seconds
Streamlit says "Connection refused"	Make sure `ollama serve` is running in another terminal
App is very slow or freezing	Switch to FastAPI mode — it uses 90% less RAM